Problemas do Dicionário Alienígena e da Escada de Palavras

Prof. Thiago Braga Marcilon

Universidade Federal do Cariri

CC0058 – Tópicos Avançados em Algoritmos e Complexidade

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

- na letra da primeira posição i em que diferem (da esquerda pra direita), $s[i] < t[i]$; ou

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

- na letra da primeira posição i em que diferem (da esquerda pra direita), $s[i] < t[i]$; ou
- se as primeiras $min(s.length, t.length)$ letras são as mesmas, então $s.length < t.length$

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

- na letra da primeira posição i em que diferem (da esquerda pra direita), $s[i] < t[i]$; ou
- se as primeiras $min(s.length, t.length)$ letras são as mesmas, então $s.length < t.length$

Dicionário Alienígena

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

- na letra da primeira posição i em que diferem (da esquerda pra direita), $s[i] < t[i]$; ou
- se as primeiras $min(s.length, t.length)$ letras são as mesmas, então $s.length < t.length$

Dicionário Alienígena

- **Entrada:** Um array A com n strings ordenadas em ordem lexicográfica do Dicionário Alienígena;

Problema do Dicionário Alienígena

Ordem lexicográfica

Uma palavra s vem antes de outra t em uma ordem lexicográfica se:

- na letra da primeira posição i em que diferem (da esquerda pra direita), $s[i] < t[i]$; ou
- se as primeiras $min(s.length, t.length)$ letras são as mesmas, então $s.length < t.length$

Dicionário Alienígena

- **Entrada:** Um array A com n strings ordenadas em ordem lexicográfica do Dicionário Alienígena;
- **Tarefa:** Retornar uma string s com todos os caracteres que aparecem em alguma string de A de forma que esses caracteres aparecem em s ordenados de acordo com a ordem lexicográfica definida pelas strings em A.

Ideias

Ideias

Ideias

Ideias

- Podemos modelar o problema com um grafo!

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo?

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!
  - Basta compararmos $A[i]$ com $A[i+1]$ p/ todo $0 \leq i < n-1$;

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!
  - Basta compararmos $A[i]$ com $A[i+1]$ p/ todo $0 \leq i < n-1$;
- Temos o grafo... e agora?

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!
  - Basta compararmos $A[i]$ com $A[i+1]$ p/ todo $0 \leq i < n-1$;
- Temos o grafo... e agora?
  - Ordenação topológica!

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!
  - Basta compararmos $A[i]$ com $A[i+1]$ p/ todo $0 \leq i < n-1$;
- Temos o grafo... e agora?
  - Ordenação topológica!
- Complexidade?

Ideias

Ideias

- Podemos modelar o problema com um grafo!
- As letras são os vértices do grafo ...
- ... e a aresta de uma letra c_1 para outra c_2 significa que c_1 vem antes de c_2 na ordem lexicográfica alienígena;
- Como construir esse grafo? Precisamos comparar cada par de palavras?
  - NÃO!
  - Basta compararmos $A[i]$ com $A[i+1]$ p/ todo $0 \leq i < n-1$;
- Temos o grafo... e agora?
  - Ordenação topológica!
- Complexidade? $O(n)$, onde n é a soma dos tamanhos de todas as palavras.

Ordenação Topológica: Algoritmo de Kahn

Require: Grafo orientado $G = (V, E)$
Ensure: Ordem topológica de V, ou Falha se G tem ciclo

```
1:  for cada v ∈ V do
2:      grau[v] ← número de arestas entrando em v
3:  Q ← fila com todos os v tais que grau[v] = 0
4:  L ← lista vazia
5:  while Q ≠ ∅ do
6:      u ← Desenfileira(Q)
7:      Anexa(L, u)
8:      for cada aresta (u, v) ∈ E do
9:          grau[v] ← grau[v] − 1
10:         if grau[v] = 0 then
11:             Enfileira(Q, v)
12: if |L| < |V| then
13:     return Falha
14: return L
```

Problema da Escada de Palavras

Escada de Palavras

Problema da Escada de Palavras

Escada de Palavras

- Um caminho de palavras é uma sequência $s_1 \to s_2 \to \ldots \to s_k$ tal que s_i difere de s_{i+1} em apenas uma posição.

Problema da Escada de Palavras

Escada de Palavras

- Um caminho de palavras é uma sequência $s_1 \to s_2 \to \ldots \to s_k$ tal que s_i difere de s_{i+1} em apenas uma posição.

Problema da Escada de Palavras

Problema da Escada de Palavras

Escada de Palavras

- Um caminho de palavras é uma sequência $s_1 \rightarrow s_2 \rightarrow \ldots \rightarrow s_k$ tal que s_i difere de s_{i+1} em apenas uma posição.

Problema da Escada de Palavras

- **Entrada:** duas palavras *beginWord* e *endWord* e um array A com n palavras;

Problema da Escada de Palavras

Escada de Palavras

- Um caminho de palavras é uma sequência $s_1 \to s_2 \to \ldots \to s_k$ tal que s_i difere de s_{i+1} em apenas uma posição.

Problema da Escada de Palavras

- **Entrada:** duas palavras *beginWord* e *endWord* e um array A com n palavras;
- **Tarefa:** Determinar se há um caminho entre *beginWord* e *endWord*, usando apenas as palavras em A no caminho;

Problema da Escada de Palavras

Escada de Palavras

- Um caminho de palavras é uma sequência $s_1 \to s_2 \to \ldots \to s_k$ tal que s_i difere de s_{i+1} em apenas uma posição.

Problema da Escada de Palavras

- **Entrada:** duas palavras *beginWord* e *endWord* e um array A com n palavras;
- **Tarefa:** Determinar se há um caminho entre *beginWord* e *endWord*, usando apenas as palavras em A no caminho;
- **Obs:** Para haver um caminho, *beginWord* não precisa estar em A, mas *endWord* sim.

Ideias

Ideias

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;
- E depois?

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;
- E depois?
  - BFS?

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;
- E depois?
  - BFS?
  - A^*!

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;
- E depois?
  - BFS?
  - A^*!
- O algoritmo A^* é bastante similar ao Dijkstra, mas as chaves na fila de prioridade não são as distâncias para a origem;

Ideias

Ideias

- Constrói o grafo onde cada palavra é um vértice e...
- ... temos uma arestas entre duas palavras se elas diferem em uma só posição;
- E depois?
  - BFS?
  - A^*!
- O algoritmo A^* é bastante similar ao Dijkstra, mas as chaves na fila de prioridade não são as distâncias para a origem;
- A chave de cada vértice v é $d(v) + h(v)$, onde $d(v)$ é a distância para a origem e $h(v)$ é uma estimativa da distância de v para o destino.

Algoritmo A*

Require: Grafo $G = (V, E)$ com pesos $w \geq 0$, origem s, destino t, heurística h
Ensure: Caminho de custo mínimo de s a t, ou Falha

```
1:  d[v] ← ∞ para todo v ∈ V;
2:  d[s] ← 0
3:  A ← fila de prioridade contendo s com chave f(s) = h(s)
4:  while A ≠ ∅ do
5:      u ← ExtraiMin(A)
6:      if u = t then
7:          return d[t]
8:      for cada aresta (u, v) ∈ E do
9:          tent ← d[u] + w(u, v)
10:         if tent < d[v] then
11:             d[v] ← tent
12:             InsereOuAtualiza(A, v, d[v] + h(v))
13: return Falha
```

Ideias

Ideias

Ideias

Ideias

- Usar sempre o A^* quando houver uma heurística *segura*, ou seja, quando $h(v, t) \leq dist(v, t)$ para todo v e destino t;

Ideias

Ideias

- Usar sempre o A^* quando houver uma heurística *segura*, ou seja, quando $h(v, t) \leq dist(v, t)$ para todo v e destino t;
- Qual heurística usar?

Ideias

Ideias

- Usar sempre o A^* quando houver uma heurística *segura*, ou seja, quando $h(v, t) \leq dist(v, t)$ para todo v e destino t;
- Qual heurística usar?
  - O número de posições diferentes entre a palavra atual e *endWord*;

Ideias

Ideias

- Usar sempre o A^* quando houver uma heurística *segura*, ou seja, quando $h(v, t) \leq dist(v, t)$ para todo v e destino t;
- Qual heurística usar?
  - O número de posições diferentes entre a palavra atual e *endWord*;
- Complexidade: $O(n + m)$ o BFS e, dependendo da E.D. concreta da fila, $O((n + m) \log m)$ ou $O(m + n \log n)$ para o A^*;

Ideias

Ideias

- Usar sempre o A^* quando houver uma heurística *segura*, ou seja, quando $h(v,t) \leq dist(v,t)$ para todo v e destino t;
- Qual heurística usar?
  - O número de posições diferentes entre a palavra atual e *endWord*;
- Complexidade: $O(n+m)$ o BFS e, dependendo da E.D. concreta da fila, $O((n+m)\log m)$ ou $O(m+n\log n)$ para o A^*;
- mas, na prática, A^* é melhor no caso médio se implementada com heap binária ou mesmo com uma heap de emparelhamento.